

Lecture 5

บท: Recursion

วิชา: 819605 Discrete Mathematics

อาจารย์ผู้สอน: ปพนธีร์ แย้มโชติ | วิศวกรรมคอมพิวเตอร์ | ภาคปลาย ปีการศึกษา 2568

เป้าหมายการเรียนรู้ของเอกสารประกอบนี้ (Week 5): เน้นที่การ solving มากกว่า proving

1. เข้าใจหลักการของแนวคิดการแก้ปัญหาด้วยวิธีการเวียนเกิด (recursion)
2. สามารถพิจารณาลักษณะของปัญหาที่สามารถแก้ด้วยการเวียนเกิดได้
3. และสามารถแก้ปัญหาดังกล่าวด้วยวิธีการเวียนเกิดได้
4. (optional) สามารถ implement การเวียนเกิดด้วยการเขียน code ได้ (สอนด้วย C (อาจจะ Python ขึ้นกับข้อสรุปในห้อง))

1 แนวคิดพื้นฐานของการเวียนเกิด

การเวียนเกิด (recursion) คือรูปแบบการแก้ปัญหา (หรือการคำนวณ) ปัญหาหนึ่ง ๆ ที่อาศัยผลลัพธ์จากการแก้ปัญหาเดียวกันแต่ input เล็กกว่านำมาประกอบขึ้นเป็นผลลัพธ์ของปัญหาที่กำลังแก้

ซึ่งในทางการเขียนโปรแกรมนั้น input ที่เล็กกว่ามักจะเป็นส่วนย่อยของ input ที่อยากแก้ปัญหา ตัวอย่างเช่น

- อยากแก้ปัญหาที่รับ array $A[1..n]$ เป็น input โดยใช้ผลลัพธ์จากโจทย์เดียวกันแต่แก้กับ $A[1..(n-1)]$ (กล่าวคือลองดูกับแค่ตัวแรกจนถึงตัวรองสุดท้ายก่อนว่าได้ผลลัพธ์เป็นอะไร แล้วค่อยนำคนสุดท้ายมาพิจารณาพร้อมผลลัพธ์ที่ได้มา)
- อยากคำนวณค่าของโจทย์หนึ่งที่รับ n เข้าไป โดยใช้ผลลัพธ์จากโจทย์เดียวกันที่คำนวณกับ $n-1$ และ $n-2$ มาแล้ว

การแก้ปัญหาทาง computational บนโลกนี้มี 2 แนวคิดหลัก

1. **Iterative programming:** ใช้ loop ที่ผู้พัฒนาตั้ง logic การแก้ปัญหาในแต่ละรอบไว้แบบชัดเจนแล้ว -- อ่านง่าย(สำหรับคนส่วนใหญ่) วิธีการแก้ปัญหาเหมือนพาจับมือทำ ภาษาโปรแกรมส่วนใหญ่ออกแบบมาเพื่อเขียนแบบนี้อยู่แล้ว แต่การได้วิธีการแก้บางโจทย์ค่อนข้างยากและไม่ตรงไปตรงมา และตรวจสอบความถูกต้องทางทฤษฎีได้ลำบาก
2. **Recursive programming:** ฟังก์ชันเรียกใช้ตัวเองเพื่อเอาคำตอบจากปัญหาที่เล็กกว่า -- บางคนบอกว่าอ่านยาก(แต่สำหรับผม code คลื่นกว่า) วิธีการได้มาของผลลัพธ์เหมือนต้องอาศัยความรู้เก่าเลยไม่รู้ว่าจะได้มายังไง ภาษาแบบ functional programming ถูกพัฒนาเพื่อเขียนแบบนี้ (เช่น Haskell, Scala) แต่ภาษาที่ iterative ก็ทำได้ แต่ว่าการคิดวิธีการแก้โจทย์ค่อนข้างชัดเจนและเป็นธรรมชาติกับวิธีคิดมากกว่า และที่สำคัญ ตรวจสอบความถูกต้องของการทำงานของโปรแกรมด้วยการพิสูจน์ทางคณิตศาสตร์ได้ (อาจารย์คิดว่าวิธีคิดของมนุษย์ใกล้เคียงการแก้ปัญหแบบ recursive มากกว่า iterative เพราะปกติมนุษย์มักใช้ประสบการณ์ในการลองแก้ปัญหาที่ง่ายกว่าก่อนแล้วค่อยพัฒนาผลลัพธ์ปัญหาย่อยให้เป็นผลลัพธ์ของปัญหาที่ยากขึ้น)

Example 1. ขอเริ่มที่ตัวอย่างที่ยังไม่ใช้การแก้ที่ปัญหา แต่เป็นเรื่องของวิธีการคำนวณแบบเวียนเกิดเพื่อให้เข้าใจรูปแบบการคำนวณแบบเวียนเกิดก่อน ซึ่งคือลำดับที่รู้จักกันในชื่อ "ลำดับเลขคณิต" (arithmetic sequence) ที่นิยามแบบคำอธิบายภาษามนุษย์ว่า "คือลำดับที่ผลต่างของพจน์ใด ๆ และพจน์ก่อนหน้าจะมีค่าคงที่เสมอในทุก ๆ พจน์" ถ้ากำหนดสัญลักษณ์สำหรับพจน์ที่ n ให้เขียนแทนด้วย $a(n)$ และสมมติให้ค่าคงที่คือ d เราจะเขียนอธิบายข้อความดังกล่าวเป็นภาษาคณิตศาสตร์ได้เป็น:

$$\forall n \in \mathbb{N}, \underline{\hspace{10cm}}$$

ถ้าสมมติให้ $a(0) = 5$ และ ค่าคงที่ที่กล่าวถึงคือ $d = 3$ จงคำนวณหา $a(7)$

Example 2. อีกตัวอย่างการคำนวณคือ Fibonacci ซึ่งมีสมการนิยามคือ

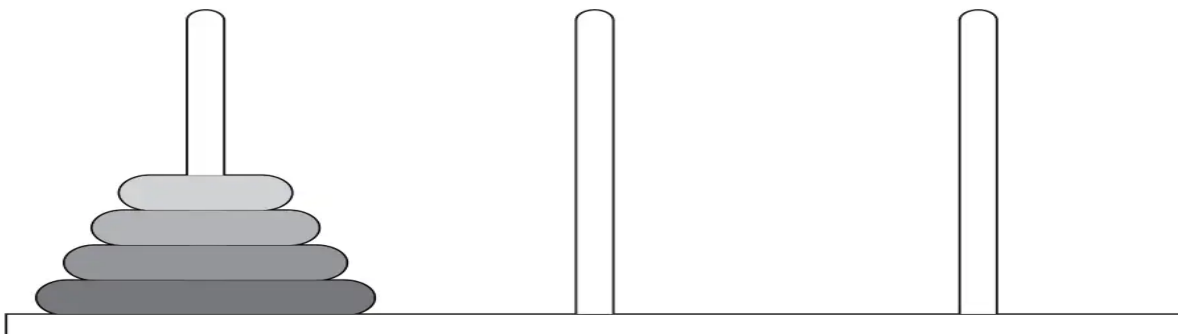
$$Fib(n) = Fib(n - 1) + Fib(n - 2)$$

โดยที่ $Fib(0) = 1$ และ $Fib(1) = 1$

จงคำนวณหา $Fib(7)$

จากตัวอย่างการคำนวณ 2 ข้อที่ผ่านมา นักศึกษาสังเกตเห็นอะไรที่เป็นส่วนสำคัญของการนิยามการคำนวณแบบเวียนเกิดบ้าง (ถามอีกแบบคือ ถ้าเอาเงื่อนไขอะไรออกจะทำให้การคำนวณเราพัง)

2 First Recursion: เกม Tower of Hanoi (เล่นอย่างไร + ต้องเล่นกี่ครั้ง)



ลองพิจารณาเกม *Tower of Hanoi* ที่มีเสา 3 ต้น (ให้เรียกชื่อว่า A, B, C) และมีแผ่นดิสก์ซ้อนกันอยู่บนเสาต้นหนึ่ง โดยมีกติกาว่า

- ย้ายได้ครั้งละ 1 แผ่นเท่านั้น
- ห้ามวางแผ่นใหญ่ทับแผ่นเล็ก (แผ่นเล็กกว่าอยู่ด้านบนของแผ่นใหญ่กว่าเสมอ)

ตอนนี้ให้สมมติสถานการณ์ดังนี้:

คุณกำลังอยากเล่น *Tower of Hanoi* ที่มี 4 แผ่นโดยนึกขึ้นได้ว่ามีเพื่อนคนหนึ่งที่มีความสามารถพิเศษว่า

ความสามารถพิเศษของเพื่อน:

สำหรับเกมที่มี 3 แผ่น และไม่ว่าจะเริ่มต้นด้วยการวางแผ่นทั้ง 3 ซ้อนกันอยู่บนเสาต้นไหนก็ตาม เพื่อนของคุณสามารถย้ายทั้ง 3 แผ่นไปยังเสาอีกต้นหนึ่ง (ที่คุณบอก) ได้ถูกต้องเสมอ โดยใช้เสาที่เหลือเป็นตัวช่วยตามกติกา

กล่าวคือ เพื่อนของคุณเป็น ``ผู้เชี่ยวชาญ Tower of Hanoi 3 แผ่น" ที่คุณสามารถสั่งได้ว่า

``ช่วยย้าย 3 แผ่นจากเสา X ไปเสา Y ให้หน่อย"

และเขาจะทำได้สำเร็จให้คุณเองโดยอัตโนมัติ

โจทย์: ตอนนี้คุณต้องการออกแบบวิธีเล่นเกม *Tower of Hanoi* แบบ 4 แผ่น โดยให้เพื่อนคนเก่งที่เล่นแบบ 3 แผ่นเป็นผู้ช่วยอยู่เบื้องหลัง เป้าหมายคือ

- เริ่มต้น: แผ่นทั้ง 4 ซ้อนกันอยู่บนเสาต้นหนึ่ง (เช่น เสา A)
- เป้าหมาย: ย้ายแผ่นทั้ง 4 ไปยังอีกเสาต้นหนึ่ง (เช่น เสา C)
- ใช้เพื่อนคนเก่ง 3 แผ่นเป็น ``ชายชุดดำ" ช่วยย้ายชุดของ 3 แผ่นเล็กตามที่คุณสั่ง (โดยที่คุณไม่จำเป็นต้องเข้าใจหรือรู้เห็นว่าเล่นอย่างไร รู้แค่คำสั่งเมื่อไหร่คุณจะได้เห็นกอง 3 แผ่นนั้นย้ายไปในที่ปรารถนา)

ให้ตอบคำถามต่อไปนี้

1. สมมติว่าเริ่มต้นมีแผ่นทั้ง 4 อยู่บนเสา A และคุณต้องการย้ายไปที่เสา C โดยใช้เสา B เป็นเสาดำช่วย
 - a) ในมุมมองของคุณ แผ่น **แผ่นล่างสุด** (แผ่นที่ใหญ่ที่สุด) ควรถูกย้ายจากเสา A ไปเสา C **เมื่อไร** ในลำดับการเล่น (ต้นเกม กลางเกม หรือท้ายเกม) และเพราะอะไร?
 - b) ไม่ว่าคุณจะทำหน้าที่ให้แผ่นใหญ่สุดอยู่ที่เสาใดก็ตาม ทำไมถึงไม่กระทบการเล่น 3 แผ่นที่เหลือของเพื่อนผู้ช่วย?
 - c) ก่อนที่คุณจะย้ายแผ่นล่างสุดจาก A ไป C คุณจำเป็นต้องจัดการกับแผ่นเล็กอีก 3 แผ่นด้านบนอย่างไรบ้าง?
2. ตอนนี้ให้คุณ **เขียนแผนการเล่น 4 แผ่นแบบทีละขั้น** โดยใช้เพื่อนคนเก่ง 3 แผ่นเป็น “ชายชุดดำ” (ไม่ต้องสนใจรายละเอียดว่าเขาย้ายอย่างไร):

แผนการเล่น Tower of Hanoi 4 แผ่น (จาก A ไป C โดยใช้ B เป็นตัวช่วย):

จากแผนข้างบน ลองเขียนสรุปให้เป็น “ขั้นตอนวิธี” (เป็นภาษาคน) สำหรับการย้าย **4 แผ่น** ว่า

“ในการย้าย Tower of Hanoi 4 แผ่นจากเสาเริ่มต้นไปยังเสาปลายทาง เราจะ”

(1) _____

(2) _____

(3) _____

(4) _____

3. (Beyond specific case) จากไอเดียการเล่น 4 แผ่นข้างต้น ถ้าต้องการออกแบบวิธีเล่นสำหรับ n แผ่น ใด ๆ เราต้องการอะไรบ้าง และจะออกแบบขั้นตอนวิธีการเล่นอย่างไร

4. (Extended Question) ในกรณีของเกม n แผ่นที่นักศึกษามีเพื่อนคนเก่งที่เล่นเกม $n - 1$ แผ่นได้เทพมาก ๆ คราวนี้นักศึกษาเกิดคำถามขึ้นมาเกี่ยวกับจำนวนตาที่ต้องใช้เล่นว่าถ้าอยากเล่นให้เทพที่สุดก็ต้องเล่นแบบที่ใช้จำนวนรอบการย้ายให้ต่ำที่สุด ถ้าสมมติว่าเพื่อนของนักศึกษาถูกฝึกให้เล่นเกม $n - 1$ แผ่นโดยใช้จำนวนรอบการย้ายแผ่นน้อยที่สุดแล้ว จงหาว่าสำหรับเกม n แผ่นนั้นจะต้องย้ายกี่รอบ

3 Second Recursion: หั่นแผ่นกระดาษได้มากที่สุดกี่ชิ้น

Example 3. สมมตินักศึกษาที่มีแผ่นกระดาษที่มีขอบเขตไม่สิ้นสุด (แผ่นระนาบ) ในโจทย์นี้ เราสนใจจำนวนชิ้นจากการลากเส้นตรงแบ่งกระดาษ โดยต้องการแบ่งให้ได้จำนวนชิ้นให้มากที่สุด ซึ่งชัดเจนกว่าถ้านักศึกษาลากเส้นตรง 1 เส้น จะแบ่งกระดาษได้ออกเป็น 2 ชิ้น และถ้ายากเส้นตรง 2 เส้นให้ตัดกัน จะแบ่งกระดาษออกเป็น 4 ชิ้น (และแบ่งได้มากที่สุดแล้วด้วย 2 เส้น) จงหาว่าถ้าเราจะตัดกระดาษด้วยเส้นตรง n เส้น (เมื่อ n เป็นจำนวนนับ) แล้วจำนวนชิ้นจากการแบ่งได้มากที่สุดจะเป็นกี่ชิ้น

4 Third Recursion: เดินขึ้นบันไดให้ใช้จำนวนก้าวที่น้อยที่สุด

Example 4. ในการเดินขึ้นบันได n ชั้น กำหนดให้เดินขึ้นได้ 2 แบบ คือ เดินขึ้นทีละ 1 ชั้น หรือทีละ 2 ชั้น จงเขียนสมการความสัมพันธ์เวียนเกิดแสดงจำนวนวิธีในการเดินขึ้นบันไดว่ามีกี่วิธีที่แตกต่างกันที่จะเดินไปชั้นที่ n

5 Fourth Recursion: จำนวนเซตย่อย

Example 5. จงหาจำนวนเซตย่อยทั้งหมดของเซตที่มีสมาชิก n ตัว โดยสมมติว่าเป็นเซต $A = \{1, 2, 3, \dots, n\}$

6 Homework: Week 5

ในสัปดาห์นี้เราจะทำแค่ส่วนการแก้ปัญหา หรือก็คือส่วนการออกแบบ แต่เราจะกลับมาที่โจทย์เหล่านี้อีกรอบเพื่อพิสูจน์ว่าวิธีการดังกล่าวถูกต้อง

1. ทุกคนทราบดีว่านิยามของการคำนวณ factorial คือ $n! = 1 \times 2 \times 3 \times \dots (n-1) \times n$ หรือถ้ากล่าวเป็นภาษาชาวบ้านคือ "ผลคูณของจำนวนนับตั้งแต่ 1 จนถึง n " เช่นตอนเป็นนักเรียนมัธยมถ้าอยากคำนวณ $5!$ เราก็จะคำนวณ $1 \times 2 \times 3 \times 4 \times 5$ ได้ผลลัพธ์เป็น 120

จงเขียนนิยามของการคำนวณ factorial ในรูปแบบ recursion โดยสมมติให้สัญลักษณ์ $Fac(n)$ แทนค่า factorial ของ n พร้อมทั้งระบุผลลัพธ์พื้นฐานเมื่อ $n = 0$

นอกจากนี้ จงเขียน code (Pseudocode หรือ syntax ภาษาที่ถนัดก็ได้) ที่แสดงการคำนวณ factorial แบบ iteration และ recursion

2. นักศึกษากำลังพิจารณาโจทย์ปัญหาการหาว่าจำนวนเต็มที่กำหนดมาให้มีอยู่ใน array ที่กำหนดมาให้หรือไม่ ถ้ามีให้คืนค่า index แรกสุดที่เจอ แต่ถ้าไม่มีให้คืน -1 ตัวอย่างเช่น ให้ $A = [1, -5, 0, 3, -5]$ และถ้าค้นหา -5 จะต้องได้ค่า return ออกมาเป็น 1 หรือถ้าค้นหา 7 จะต้องได้ค่า return ออกมาเป็น -1

แน่นอนว่าวิธีการหนึ่งที่นักศึกษาทุกคนต้องเคยทำมาแล้วคือการวนลูปตาม index แต่ในคราวนี้ให้นักศึกษาอธิบายวิธีการแก้ปัญหานี้ด้วยแนวคิด recursion โดยสมมติว่านักศึกษากำลังออกแบบวิธีคิดของฟังก์ชัน $FindFirst(arr, x)$ โดยที่ arr เป็น array และ x คือสมาชิกที่อยากค้นหาใน arr

3. ในบริบทเดียวกันกับข้อ 2 แต่คราวนี้ขอให้ทำ $FindLast(arr, x)$ กล่าวคือค้นหา index สุดท้ายที่ x ปรากฏใน arr เช่น $FindLast([1, -5, 0, 3, -5], -5)$ ต้องได้ 4

(หมายเหตุ: กันคนหัวทอม - ห้าม reverse แล้วใช้ $FindFirst$)

4. สมมติให้นักศึกษาอยากนิยามเซตของ bit strings (ตัวอักษรที่สร้างมาจากเลข 0 หรือ 1) ที่มีคุณสมบัติว่าห้ามมีเลข 0 อยู่ติดกันเกิน 2 ตัว กล่าวคืออยากได้เซต

$$A = \{0, 1, 00, 01, 10, 11, 001, 010, 011, 100, 101, 110, 111, 0010, \dots\}$$

กล่าวคือสมาชิกที่หายไปจากเซตที่เขียนไว้คือ 000, 0000, 0001 เป็นต้น

แต่แน่นอนการจะเขียนโปรแกรมให้คอมพิวเตอร์ช่วยสร้าง bit strings นี้ด้วย for loop อาจจะต้องมีการควบคุม flow ที่มีเงื่อนไขลูกจิก (แต่ยังทำได้จริงอยู่) คราวนี้เลยอยากออกแบบวิธีสร้างแบบ recursion โดยสมมติว่าจะแบ่งการนิยามออกเป็นตามความยาว $A = A_1 \cup A_2 \cup A_3 \cup \dots$ โดยที่ A_n คือเซตของ bit strings ความยาว n ที่สอดคล้องเงื่อนไขที่เรา กำลังสนใจ กล่าวคือ

$$A_1 = \{0, 1\}, A_2 = \{00, 01, 10, 11\}, A_3 = \{001, 010, 011, 100, 101, 110, 111\}$$

$$A_4 = \{0010, 0011, 0100, 0101, 0110, 0111, 1001, 1010, 1001, 1100, 1101, 1110, 1111\}$$

จงนิยาม A_n โดย recursion (คำใบ้: ต้องใช้ A_{n-2} มาช่วยนิยาม และนิยาม A_1 และ A_2 เป็นพื้นฐาน)

7 เผลยการบ้าน

1. ทุกคนทราบดีว่านิยามของการคำนวณ factorial คือ $n! = 1 \times 2 \times 3 \times \dots \times (n-1) \times n$ หรือถ้ากล่าวเป็นภาษาชาวบ้านคือ "ผลคูณของจำนวนนับตั้งแต่ 1 จนถึง n " เช่นตอนเป็นนักเรียนมัธยมถ้าอยากคำนวณ $5!$ เราก็จะคำนวณ $1 \times 2 \times 3 \times 4 \times 5$ ได้ผลลัพธ์เป็น 120

จงเขียนนิยามของการคำนวณ factorial ในรูปแบบ recursion โดยสมมติให้สัญลักษณ์ $Fac(n)$ แทนค่า factorial ของ n พร้อมทั้งระบุผลลัพธ์ขั้นฐานเมื่อ $n = 0$

นอกจากนี้ จงเขียน code (Pseudocode หรือ syntax ภาษาที่ถนัดก็ได้) ที่แสดงการคำนวณ factorial แบบ iteration และ recursion

แนวคิด: $n! = 1 \times 2 \times 3 \times \dots \times (n-1) \times n = (1 \times 2 \times 3 \times \dots \times (n-1)) \times n = (n-1)! \times n$
ดังนั้น เราจึงสามารถเขียนนิยาม factorial ในรูปแบบ recursion ได้เป็น $Fac(n) = F(n-1) \times n$ โดยที่ $Fac(0) = 1$

หมายเหตุ: สาเหตุที่ทำให้เราต้องนิยาม $Fac(0) = 1$ เพราะเราต้องการใช้ $1 = Fac(1) = Fac(0) \times 1$

Algorithm 1 Factorial แบบ iteration

Require: จำนวนนับ $n \geq 0$

```
1:  $fac = 1$   $\triangleright fac(0) = 1$ 
2: for  $i = 1..n$  do
3:    $fac = fac \times i$ 
4: end for
5: return  $fac$ 
```

Algorithm 2 $fac(n)$: Factorial แบบ recursion

Require: จำนวนนับ $n \geq 0$

```
1: if  $n == 0$  then
2:   return 1  $\triangleright fac(0) = 1$ 
3: else
4:   return  $fac(n-1) \times n$ 
5: end if
```

2. นักศึกษากำลังพิจารณาโจทย์ปัญหาการหาว่าจำนวนเต็มที่กำหนดมาให้มีอยู่ใน array ที่กำหนดมาให้หรือไม่ ถ้ามีให้คืนค่า index แรกสุดที่เจอ แต่ถ้าไม่มีให้คืน -1 ตัวอย่างเช่น ให้ $A = [1, -5, 0, 3, -5]$ และถ้าค้นหา -5 จะต้องได้ค่า return ออกมาเป็น 1 หรือถ้าค้นหา 7 จะต้องได้ค่า return ออกมาเป็น -1

แน่นอนว่าวิธีการหนึ่งที่นักศึกษาทุกคนต้องเคยทำมาแล้วคือการวนลูปตาม index แต่ในคราวนี้ให้นักศึกษาอธิบายวิธีการแก้ปัญหานี้ด้วยแนวคิด recursion โดยสมมติว่านักศึกษากำลังออกแบบวิธีคิดของฟังก์ชัน FindFirst(arr,x) โดยที่ arr เป็น array และ x คือสมาชิกที่อยากค้นหาใน arr

เพื่อใครยังไม่รู้แม้กระทั่งการทำ loop

Algorithm 3 FindFirst แบบ iteration

Require: อาร์เรย์ของจำนวนเต็ม arr และจำนวนเต็ม x

```
1: for idx = 0..(len(arr)-1) do
2:   if x == arr[idx] then
3:     return idx
4:   end if
5: end for
6: return -1
```

และจะสามารถเขียนแบบ recursion ได้ดังนี้

Algorithm 4 FindFirst(arr,x): FindFirst แบบ recursion

Require: อาร์เรย์ของจำนวนเต็ม arr และจำนวนเต็ม x

```
1: if len(arr) == 0 then
2:   return -1
3: else
4:   induct_result = FindFirst(arr[1:], x)
5:   if x == arr[0] then
6:     return 0
7:   else if induct_result == -1 then
8:     return -1
9:   else
10:    return 1 + induct_result
11:   end if
12: end if
```

3. ในบริบทเดียวกันกับข้อ 2 แต่คราวนี้ขอให้ทำ FindLast(arr,x) กล่าวคือค้นหา index สุดท้ายที่ x ปรากฏใน arr เช่น FindLast([1, -5, 0, 3, -5], -5) ต้องได้ 4
(หมายเหตุ: กันคนหัวหมอ - ห้าม reverse แล้วใช้ FindFirst)

Algorithm 5 FindLast(arr,x): FindLast แบบ recursion

Require: อาร์เรย์ของจำนวนเต็ม arr และจำนวนเต็ม x

```

1: if len(arr) == 0 then
2:     return -1
3: else
4:     induct_result = FindLast(arr[0..(len(arr)-2)], x)
5:     if x == arr[len(arr)-1] then
6:         return len(arr)-1
7:     else if induct_result == -1 then
8:         return -1
9:     else
10:        return induct_result
11:    end if
12: end if

```

4. สมมตินักศึกษาอยากนิยามเซตของ bit strings (ตัวอักษรที่สร้างมาจากเลข 0 หรือ 1) ที่มีคุณสมบัติว่าห้ามมีเลข 0 อยู่ติดกันเกิน 2 ตัว กล่าวคืออยากได้เซต

$$A = \{0, 1, 00, 01, 10, 11, 001, 010, 011, 100, 101, 110, 111, 0010, \dots\}$$

กล่าวคือสมาชิกที่หายไปจากเซตที่เขียนไว้คือ 000, 0000, 0001 เป็นต้น

แต่แน่นอนการจะเขียนโปรแกรมให้คอมพิวเตอร์ช่วยสร้าง bit strings นี้ด้วย for loop อาจจะต้องมีการควบคุม flow ที่มีเงื่อนไขจุกจิก (แต่ยังทำได้จริงอยู่) คราวนี้เลยอยากออกแบบวิธีสร้างแบบ recursion โดยสมมติว่าจะแบ่งการนิยามออกเป็นตามความยาว $A = A_1 \cup A_2 \cup A_3 \cup \dots$ โดยที่ A_n คือเซตของ bit strings ความยาว n ที่สอดคล้องเงื่อนไขที่เรากำลังสนใจ กล่าวคือ

$$A_1 = \{0, 1\}, A_2 = \{00, 01, 10, 11\}, A_3 = \{001, 010, 011, 100, 101, 110, 111\}$$

$$A_4 = \{0010, 0011, 0100, 0101, 0110, 0111, 1001, 1010, 1011, 1100, 1101, 1110, 1111\}$$

จึงนิยาม A_n โดย recursion (คำใบ้: ต้องใช้ A_{n-2} มาช่วยนิยาม และนิยาม A_1 และ A_2 เป็นขั้นฐาน)

สาเหตุที่ต้องนิยามโดยอาศัย A_{n-2} (ก่อนหน้า 2 ขั้น) เพราะการดูแค่ 1 ตัวก่อนหน้าไม่เพียงพอในการนิยามด้วย 0 เพราะถ้าเป็น $\dots 0$ เราจะไม่สามารถต่อ 0 ได้ แต่ถ้าเป็น $\dots 1$ เราจะสามารถต่อได้ทั้ง 0 และ 1 ซึ่งทำได้โดยการดูว่าสมาชิก $x \in A_{n-2}$ นั้นมี 2 ตัวท้ายเป็นอะไร ถ้าลงท้ายด้วย 00 จะเติมได้แค่ 10 และ 11 แต่ถ้าลงท้ายด้วย 10 จะเติม 01, 10, 11 ได้ ในขณะที่ถ้าลงท้ายด้วย 1 จะเติมต่อได้หมด ดังนั้น เราจะนิยามแบบ recursion ได้เป็น

$$A_n = \{x10, x11 \mid x[-2]00 \in A_{n-2}\} \cup \{x01, x10, x11 \mid x[-2]10 \in A_{n-2}\} \cup \{x00, x01, x10, x11 \mid x[-1]1 \in A_{n-2}\}$$

โดย $A_1 = \{0, 1\}, A_2 = \{00, 01, 10, 11\}$

สำหรับนักศึกษาที่คิดว่าอ่านโค้ดง่ายกว่าอ่านคณิตศาสตร์ แต่ทั้งนี้โค้ดที่เขียนมายังไม่เป็นรูปแบบ pure functional (ยังเหลือ for loop อยู่เพื่อให้นักศึกษาอ่านเข้าใจง่ายแบ่งตามเซตที่แบ่งยูเนียนไว้ // เพราะลองเขียนเป็น pure functional แล้วรู้สึกยากไปนิดสำหรับนักศึกษา):

```
def create_at_rec(n):
    if n == 1:
        return {"0", '1'}
    elif n == 2:
        return {'00', '01', '10', '11'}
    elif n == 3:
        return {'001', '010', '011', '100', '101', '110', '111'}
    result = set()
    base = create_at_rec(n-2)
    for x in base:
        if x.endswith('00'):
            result = result.union({x+'10', x+'11'})
        elif x.endswith('1'):
            result = result.union({x+'00', x+'01', x+'10', x+'11'})
        elif x.endswith('10'):
            result = result.union({x+'01', x+'10', x+'11'})
    return result
```

อันนี้เป็น Pure Functional ใน Haskell สำหรับนักศึกษาที่อยากลอง challenge ตัวเอง:

```
import Data.List (isSuffixOf)

suffix2 :: String -> String
suffix2 x = drop (length x - 2) x

allowed :: String -> [String]
allowed s = case s of
    "00" -> ["10", "11"]
    "10" -> ["01", "10", "11"]
    "01" -> ["00", "01", "10", "11"]
    "11" -> ["00", "01", "10", "11"]
    _    -> error "unexpected suffix"

extend :: String -> [String]
extend x = [ x ++ t | t <- allowed (suffix2 x) ]
```

อีกไอเดียเพื่อเขียนโปรแกรมแบบ pure functional เราจะแบ่งการนิยามออกเป็นเซตย่อย 3 ตามส่วนหลัง (เหมือนที่เขียน for loop ไว้ด้านบน) ทว่า เราต้องแก้ปัญหการเขียน for loop ด้วยการสร้าง function สำหรับการเติมอักขรต่อท้ายเข้าไปก่อนแทนการใช้ for loop ซึ่งใน code ด้านล่างคือฟังก์ชัน append_all นอกจากนี้ เรายังต้องการฟังก์ชันที่ใช้ในการแยกชิ้นส่วนของเซตปัจจุบันออกเป็น 3 เซตย่อย กล่าวคือเราต้องการ

$$A_n = A_n^{(1)} \cup A_n^{(00)} \cup A_n^{(10)}$$

โดยที่ $A_n^{(x)}$ คือเซตของ bit string ความยาว n ที่เราต้องการ (ไม่มีเลข 0 ติดกันเกิน 2 ตัว) และลงท้ายด้วย x จึงต้องสร้างฟังก์ชันชื่อ get_end_with เพื่อที่จะแยกออกมาเป็น 3 เซตดังกล่าว ซึ่งใน code คือ base_1, base_00 และ base_10 เพื่อนำมา append ตามที่ได้กล่าวไว้

และเมื่อนำมารวมกัน จะเขียนฟังก์ชันเพื่อสร้างเซตดังกล่าวแบบ recursion ที่อาศัยแต่ pure function ได้ดังนี้

```

1 def append_all(A,x):
2     if len(A) == 0: return []
3     else: return [A[0] + x] + append_all(A[1:],x)
4
5 def get_end_with(A, x):
6     if len(A) == 0: return []
7     elif len(A) == 1: return [A[0]] if A[0].endswith(x) else []
8     else: #len(A) >= 2
9         result = get_end_with([A[0]],x)
10        return result + get_end_with(A[1:], x)
11
12 def create_at_functional(n):
13     if n == 0: return []
14     elif n == 1: return ['0','1']
15     elif n == 2: return ['00','01','10','11']
16     elif n == 3: return ['001','010','011','100','101','110','111']
17     else:
18         base = create_at_functional(n-2)
19         base_1 = get_end_with(base, '1')
20         base_00 = get_end_with(base, '00')
21         base_10 = get_end_with(base, '10')
22         result_1 = append_all(base_1, '10') + append_all(base_1, '11') + append_all(base_1, '00') + append_all(base_1, '01')
23         result_00 = append_all(base_00, '10') + append_all(base_00, '11')
24         result_10 = append_all(base_10, '10') + append_all(base_10, '11') + append_all(base_10, '01')
25         return result_1 + result_00 + result_10
26
27 create_at_functional(6)

```